WEEK 1 · LESSON 3 OF 24 · HANDOUT

Serving a Model with FastAPI

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=mMQ6mGQtwX8 — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon India 2021 keynote: Modern Python through FastAPI and friends (Sebastián Ramírez) — https://www.youtube.com/watch?v=3qTBLhnS4to

🎓 **Go deeper:** EuroPython: Train, Serve, Deploy — an NLP model with PyTorch and Docker — https://www.youtube.com/watch?v=BSDW8PWUVF0

Your goal in this lesson: Serve a trained model as a web API with FastAPI, validate inputs automatically, return versioned answers, support batch requests, and test the API in code.

Why an API?

A web or mobile app, a loan officer's dashboard or a USSD menu cannot run your notebook. They send an **HTTP request** with the applicant's data and get back JSON. FastAPI is a popular Python library for this: fast, simple, and it checks inputs for you using **pydantic** models. (FastAPI is already installed in Google Colab.)

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

```
import joblib
joblib.dump(model, "loan_model.joblib")
MODEL_VERSION = "1.0.0"
```

Describe the input once

```
from pydantic import BaseModel, Field

class Applicant(BaseModel):
    income_rwf: float = Field(..., ge=10_000, le=20_000_000, description="Monthly income in
RWF")
    loan_rwf: float = Field(..., ge=10_000, le=5_000_000)
    years_member: int = Field(..., ge=0, le=60)
    late_payments: int = Field(..., ge=0, le=50)
    salaried: int = Field(..., ge=0, le=1)
    momo_tx_month: int = Field(..., ge=0, le=1000)

class Decision(BaseModel):
    repay_probability: float
    approve: bool
    model_version: str
```

`ge` means "greater than or equal", `le` "less than or equal". Any request outside these limits is rejected **before** the model sees it — the input checks from Lesson 1, written once.

Notice what is **not** in `Applicant`: gender, area, name, phone number. The model does not need them, so the API does not accept them. Collecting less data is safer (Week 3).

The API

```
from fastapi import FastAPI

app = FastAPI(title="SACCO loan scoring", version=MODEL_VERSION)
scorer = joblib.load("loan_model.joblib")          # load once at start-up, not on every
request
THRESHOLD = 0.5

@app.get("/health")
def health():
    return {"status": "ok", "model_version": MODEL_VERSION}

@app.post("/predict", response_model=Decision)
def predict(applicant: Applicant):
    row = pd.DataFrame([applicant.model_dump()])[FEATURES]
    p = float(scorer.predict_proba(row)[0, 1])
    return Decision(repay_probability=round(p, 3), approve=p >= THRESHOLD,
model_version=MODEL_VERSION)

@app.post("/predict/batch", response_model=list[Decision])
def predict_batch(applicants: list[Applicant]):
    rows = pd.DataFrame([a.model_dump() for a in applicants])[FEATURES]
    ps = scorer.predict_proba(rows)[:, 1]
    return [Decision(repay_probability=round(float(p), 3), approve=bool(p >= THRESHOLD),
model_version=MODEL_VERSION)
            for p in ps]
```

Every answer includes the **model version**, so later you know which model made each decision.

Test it without a server

FastAPI's `TestClient` sends real HTTP requests to the app inside your notebook:

```
from fastapi.testclient import TestClient

client = TestClient(app)
print(client.get("/health").json())

applicant = {"income_rwf": 350_000, "loan_rwf": 200_000, "years_member": 5,
             "late_payments": 0, "salaried": 1, "momo_tx_month": 40}
r = client.post("/predict", json=applicant)
print(r.status_code, r.json())

bad = dict(applicant, income_rwf=350)
r = client.post("/predict", json=bad)
print(r.status_code, r.json()["detail"][0]["msg"])

r = client.post("/predict/batch", json=[applicant, dict(applicant, late_payments=4,
loan_rwf=900_000)])
print(r.status_code, r.json())
```

- A good request returns **200** with the decision.
- The income of 350 RWF returns **422 Unprocessable Entity** with a clear message — the model never ran.
- The batch endpoint scores many applicants in one call (faster than one call each).

Run it for real

Save the code as `main.py` and start it on your computer or server:

```
pip install fastapi uvicorn scikit-learn pandas joblib
uvicorn main:app --host 0.0.0.0 --port 8000
```

Then open `http://localhost:8000/docs`: FastAPI creates an interactive documentation page where you can try every endpoint.

Good serving habits

- **Load the model once** at start-up, not on every request.
- **Validate inputs** with pydantic; accept only the fields you need.
- **Version every answer**.
- **Never log personal data** in plain text (Week 3).
- **Add a /health endpoint** so your server, Docker and monitoring can check the service is alive.

Key terms

- **API** — a way for programs to talk over HTTP.
- **Endpoint** — one URL and method, such as POST /predict.
- **pydantic model** — a class that describes and validates data.
- **422** — the status code for input that fails validation.
- **TestClient** — sends requests to a FastAPI app inside Python, for testing.
- **Batch endpoint** — scores many items in one request.

Remember this: FastAPI turns a model into a validated, versioned, testable web service in a few dozen lines — load once, check every input, version every answer.

Name: _____ Date: _____

Activity

Task 1

Send a request with `salaried` set to 2. Read the error message.

Task 2

Add a field `reason` to `Decision` that says "high loan-to-income" when the loan is more than 3 times the income.

Task 3

Add a `GET /model` endpoint that returns the features and threshold.

Task 4

Time 100 single requests and one batch request of 100. Which is faster?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does FastAPI use to check request data automatically?
   - A. Docker
   - B. SQL
   - C. pydantic models
   - D. pandas
2. Which status code does FastAPI return for input that fails validation?
   - A. 500
   - B. 200
   - C. 422
   - D. 404
3. Why load the model at start-up?
   - A. Loading on every request would be slow
   - B. It is required by law
   - C. To hide it
   - D. To change it each time
4. Why return the model version with each answer?
   - A. To slow down attackers
   - B. So you later know which model made each decision
   - C. For decoration
   - D. It improves accuracy
5. Why does the Applicant model not accept gender or phone number?
   - A. FastAPI forbids it
   - B. The model does not need them, and collecting less personal data is safer
   - C. They are too long
   - D. They are always missing
6. What does TestClient let you do?
   - A. Train the model
   - B. Send real HTTP requests to the app inside Python, for testing
   - C. Write documentation
   - D. Deploy to the cloud

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.